Modelado del Almacén como Dígrafo de Pasillos sobre una Tabla Hash Universal Propia

Representación Estructural, Algoritmo de Construcción, Validación por Componentes Fuertemente Conexas y Análisis de Complejidad

Cátedra de Algoritmos y Estructuras de Datos II
Proyecto Semestral: Simulación de Wave Picking en Warehouse

Abstract

Este documento formaliza la capa de modelado del sistema: la traducción del escenario descripto en JSON a un *dígrafo de pasillos* $G = (V, E)$ sobre el cual operan el armado de waves, el cálculo de rutas y la simulación. Se definen con precisión el conjunto de nodos (celdas transitables), la regla que genera las aristas dirigidas a partir del sentido de circulación, y el cálculo de las celdas de acceso a racks y zonas de despacho. Toda la topología se almacena en una *tabla hash propia* con hashing universal de Carter y Wegman, resolución de colisiones por encadenamiento y redimensionamiento a tamaño primo. La transitabilidad del mapa se certifica con el algoritmo de Kosaraju en versión iterativa. Se demuestran la inyectividad de la codificación de coordenadas, la universalidad de la familia de funciones hash, el costo esperado constante de las operaciones de la tabla, la correctitud del dígrafo y de Kosaraju, y la linealidad del proceso completo en el área del almacén. Se incluye una taxonomía de casos de borde y mediciones sobre los escenarios del proyecto.

Contents

1 Introducción y Principios de Diseño **2**
1.1 Entidades y Clases Involucradas 2

2 Representación Formal del Almacén **3**
2.1 Elección de la Representación 3
2.2 Accesos a Racks y Zonas de Despacho 4
2.3 Tercera Dimensión 4

3 Tabla Hash Universal Propia **4**
3.1 Estructura Interna 4
3.2 Función Hash en Dos Etapas 4
3.3 Resolución de Colisiones por Encadenamiento 5
3.4 Parámetros 5

4 Algoritmo de Construcción **5**
4.1 Ejemplo Desarrollado 6

5 Validación por Componentes Fuertemente Conexas **7**

6 Taxonomía de Casos de Construcción **9**
6.1 Caso 1: Cruce de Pasillos 9
6.2 Caso 2: Piso Libre no Declarado 9
6.3 Caso 3: Obstáculo sobre un Pasillo o sobre Otro Obstáculo 9
6.4 Caso 4: Forma del Rack 9
6.5 Caso 5: Trampas e Islas 10
6.6 Caso 6: Producto o Zona sin Acceso 10
6.7 Caso 7: Picker sobre un Obstáculo 10
6.8 Caso 8: Datos Fuera de Rango o Mal Formados 10

7 Demostraciones y Análisis Asintótico **10**
7.1 Demostración 1: Inyectividad de la Codificación de Coordenadas 10
7.2 Demostración 2: Universalidad de la Familia $h_{a,b}$ 10
7.3 Demostración 3: Costo Esperado Constante con Encadenamiento 11
7.4 Demostración 4: Correctitud del Dígrafo Construido 11
7.5 Demostración 5: Correctitud y Costo de Kosaraju 12
7.6 Demostración 6: Linealidad de la Construcción Completa 12

8 Resultados Experimentales **12**

9 Cuadro Resumen de Complejidades Computacionales **13**

10 Conclusión **14**

1 Introducción y Principios de Diseño

El resto del sistema (armado de waves, asignación de pickers, rutas y simulación) necesita responder, millones de veces, preguntas del tipo *"¿a qué celdas puedo moverme desde acá?"* o *"¿dónde frena el picker para retirar este producto?"*. La capa de modelado existe para que esas preguntas se respondan en tiempo constante esperado y para garantizar, antes de simular, que el mapa es transitable.

Responsabilidades de la capa de modelado

1. **Traducción:** convertir los rectángulos del JSON (pasillos, racks, zonas, puertas) en un dígrafo de celdas, con el sentido de circulación codificado en la dirección de las aristas.
2. **Precomputación:** calcular una sola vez, al construir, la celda de pasillo desde la que se accede a cada celda de rack y a cada zona de despacho.
3. **Certificación:** verificar con componentes fuertemente conexas que desde cualquier pasillo se pueda llegar a cualquier otro y volver, y emitir advertencias en caso contrario, sin abortar la carga.
4. **Independencia de librerías:** todas las estructuras principales del mapa (nodos, lista de adyacencia, obstáculos y grafo transpuesto) se implementan con una tabla hash propia, sin usar `dict` ni librerías de grafos.

1.1 Entidades y Clases Involucradas

- `Warehouse`: el almacén de $W \times H$ celdas (el eje y crece hacia arriba). Contiene el dígrafo, la tabla de obstáculos, los racks, zonas y puertas, el catálogo de productos, los pedidos, los pickers, los accesos precalculados y la lista de advertencias.
- `Digrafo`: dos tablas hash propias indexadas por coordenada, `nodos`: $(x, y) \mapsto$ `NodoGrafo` y `lista_adyacencia`: $(x, y) \mapsto$ lista de sucesores. Además guarda las reservas espacio-temporales que usa la simulación.
- `NodoGrafo`: una celda de pasillo con su coordenada, su sentido permitido, el identificador del pasillo que la declaró (o ninguno) y las listas de celdas de rack y zonas adyacentes desde las que es acceso.
- `Celda` y `CeldaRack`: celdas no transitables (rack, zona de despacho o puerta). La `CeldaRack` contiene `productos_en_z`: nivel $z \mapsto$ producto, que modela la tercera dimensión $\langle x, y, z \rangle$.

- `TablaHashUniversal`: la tabla hash propia, con la misma interfaz que un diccionario para no acoplar el resto del sistema a su implementación.
- `GraphBuilder`: el constructor; contiene el algoritmo de la Sección 4, Kosaraju y la validación.

2 Representación Formal del Almacén

Definición 1 (Grilla, obstáculos y nodos)**.** Sea $\Omega = \{0, \dots, W-1\} \times \{0, \dots, H-1\}$ el conjunto de celdas. Sea $\mathcal{O} \subseteq \Omega$ el conjunto de celdas ocupadas por racks, zonas de despacho o puertas. El conjunto de nodos del dígrafo es

$$V = \Omega \setminus \mathcal{O},$$

es decir, **toda celda que no es obstáculo es un nodo**, haya sido declarada como pasillo o no.

Definición 2 (Sentido de una celda)**.** Cada nodo tiene un sentido $\sigma(c) \in \{\mathsf{N}, \mathsf{S}, \mathsf{E}, \mathsf{O}, \mathsf{AMBOS}\}$. Si algún pasillo del JSON cubre a c, $\sigma(c)$ es el sentido declarado, con la regla de cruce de la Sección 4. Si ningún pasillo cubre a c, entonces $\sigma(c) = \mathsf{AMBOS}$: es piso libre que admite circulación en cualquier dirección.

Sea $\Delta = \{(0,1), (0,-1), (1,0), (-1,0)\}$ el conjunto de movimientos ortogonales (arriba, abajo, derecha, izquierda) y sea $\vec{d}(\mathsf{N}) = (0,1)$, $\vec{d}(\mathsf{S}) = (0,-1)$, $\vec{d}(\mathsf{E}) = (1,0)$, $\vec{d}(\mathsf{O}) = (-1,0)$ la dirección de avance de cada mano única.

Definición 3 (Salidas permitidas y contramano)**.** Las direcciones en las que se puede *salir* de un nodo son

$$D(\mathsf{AMBOS}) = \Delta, \qquad D(s) = \Delta \setminus \{-\vec{d}(s)\} \quad \text{para } s \in \{\mathsf{N}, \mathsf{S}, \mathsf{E}, \mathsf{O}\}.$$

Es decir, una mano única permite avanzar y doblar hacia los costados, pero no retroceder. *Entrar* a un nodo de sentido s' con el movimiento δ es contramano si

$$\mathrm{CM}(\delta, s') \iff s' \neq \mathsf{AMBOS} \ \wedge \ \delta = -\vec{d}(s').$$

Definición 4 (Aristas)**.**

$$E = \big\{ (u,\, u+\delta) \ : \ u \in V,\ \delta \in D(\sigma(u)),\ u+\delta \in V,\ \neg\, \mathrm{CM}\big(\delta, \sigma(u+\delta)\big) \big\}.$$

Toda arista tiene costo unitario $w(e) = 1$ (un paso del picker).

La doble condición es esencial: la primera la impone la celda de origen (*¿puedo salir hacia allá?*) y la segunda la celda de destino (*¿puedo entrar viniendo desde acá?*). Por ejemplo, desde una celda AMBOS ubicada debajo de un pasillo S no existe la arista hacia arriba, aunque el origen admita cualquier dirección.

2.1 Elección de la Representación

- **Dígrafo y no grafo:** los pasillos de mano única hacen que $(u,v) \in E$ no implique $(v,u) \in E$. Un grafo no dirigido no puede representarlo.
- **Lista de adyacencia y no matriz:** cada nodo tiene a lo sumo cuatro sucesores, así que $|E| \leq 4|V|$ y el grafo es ralo. La lista ocupa $\Theta(|V| + |E|) = \Theta(|V|)$ contra $\Theta(|V|^2)$ de la matriz. En el escenario grande ($|V| = 1898$, $|E| = 6048$) una matriz tendría más de 3,6 millones de entradas. Además, la operación central de los algoritmos de ruteo, enumerar los sucesores de un nodo, cuesta $O(1 + \text{grado})$.
- **Indexación por coordenada:** la identidad natural de una celda es (x, y). Indexar las tablas por coordenada permite pasar del JSON (o de un clic en la interfaz) al nodo en tiempo constante esperado, sin tablas de traducción adicionales.
- **Obstáculos fuera del grafo:** racks, zonas y puertas no son nodos, porque el picker no los atraviesa. Se guardan en una tabla aparte, $\mathcal{O}$, para poder consultar qué hay en cada celda.

2.2 Accesos a Racks y Zonas de Despacho

Un rack R con esquina (x_R, y_R), ancho a_R y alto h_R tiene un lado de longitud 2 (su profundidad). La mercadería se retira desde los *costados largos*, nunca desde las puntas. Para una celda $c = (c_x, c_y)$ del rack, el conjunto de direcciones de acceso es

$$\mathcal{D}_R(c) = \begin{cases} \{(-1,0)\} \text{ si } c_x = x_R, \quad \{(1,0)\} \text{ si no} & \text{si } a_R = 2,\ h_R \neq 2 \quad \text{(rack vertical)} \\ \{(0,-1)\} \text{ si } c_y = y_R, \quad \{(0,1)\} \text{ si no} & \text{si } h_R = 2,\ a_R \neq 2 \quad \text{(rack horizontal)} \\ \Delta & \text{si } a_R = h_R = 2. \end{cases}$$

Los accesos de c son $\mathrm{Acc}(c) = \{\, c + \delta : \delta \in \mathcal{D}_R(c) \,\} \cap V$. El primer acceso encontrado se registra como *acceso principal* en `acceso_rack_a_pasillo`, y luego la validación (Sección 5) lo reemplaza, si hace falta, por uno que pertenezca a la red principal. Para una zona de despacho Z, sus accesos son todos los nodos ortogonalmente adyacentes a alguna de sus celdas.

Con esto, la pregunta *"¿dónde frena el picker para retirar el producto p?"* se resuelve con dos accesos de costo constante: catálogo $p \mapsto (x, y)$ y luego `acceso_rack_a_pasillo`$[(x, y)]$.

2.3 Tercera Dimensión

El dígrafo vive en el plano $z = 0$: el picker solo se mueve por el piso. La altura se modela dentro de cada celda de rack con `productos_en_z`, que asocia a cada nivel $z \geq 1$ el producto almacenado allí. Así, la posición $\langle x, y, z \rangle$ de un producto determina su celda de rack, su nivel y, mediante el acceso, su nodo de recolección.

3 Tabla Hash Universal Propia

Las cuatro estructuras que guardan la topología del mapa (`nodos`, `lista_adyacencia`, `obstaculos` y el grafo transpuesto de Kosaraju) usan `TablaHashUniversal`, implementada en `BackEnd/data_structures/tabla_hash.py`.

3.1 Estructura Interna

- Un arreglo de m casillas, con m primo ($m_0 = 17$). Cada casilla apunta a una **lista simplemente enlazada** de eslabones `NodoCadena`⟨clave, índice, siguiente⟩.
- Un arreglo `_entradas` con los pares [clave, valor] **en orden de inserción**. El eslabón guarda el índice de su par en este arreglo.
- La cantidad de claves n, y dos parámetros $a \in \{1, \ldots, p-1\}$ y $b \in \{0, \ldots, p-1\}$ elegidos al azar una sola vez, con semilla fija.

3.2 Función Hash en Dos Etapas

Etapa 1: codificación de la clave como entero. Para una coordenada,

$$k(x, y) = x \cdot B + y, \qquad B = 1\,000\,003. \tag{1}$$

Para un texto $s_1 \cdots s_\ell$ se usa el polinomio $k = \left(\sum_i \mathrm{ord}(s_i)\, 131^{\ell - i}\right) \bmod p$, y para tuplas generales se combina cada componente con la base B.

Etapa 2: hashing universal de Carter y Wegman.

$$h_{a,b}(k) = \big((a\,k + b) \bmod p\big) \bmod m, \qquad p = 2^{61} - 1. \tag{2}$$

p es un primo de Mersenne mayor que cualquier clave codificada; m es la cantidad de casillas.

3.3 Resolución de Colisiones por Encadenamiento

Algoritmo 1 Operaciones de la tabla hash

```
1: función BUSCAR(T, clave)
2:     e ← T.casillas[h_{a,b}(k(clave))]
3:     mientras e ≠ NIL y e.clave ≠ clave hacer
4:         e ← e.siguiente
5:     devolver e                                   ▷ NIL si la clave no está
6: procedimiento INSERTAR(T, clave, valor)
7:     e ← BUSCAR(T, clave)
8:     si e ≠ NIL entonces T.entradas[e.indice].valor ← valor; devolver
9:     agregar [clave, valor] al final de T.entradas
10:    i ← h_{a,b}(k(clave)); T.casillas[i] ← NODOCADENA(clave, |T.entradas|−1, T.casillas[i])
       ▷ al frente, O(1)
11:    T.n ← T.n + 1
12:    si T.n > 0,75 · T.m entonces REHASH(T, SIGUIENTEPRIMO(2 T.m))
13: procedimiento REHASH(T, m′)
14:    T.entradas ← entradas no borradas, en el mismo orden; T.m ← m′; casillas vacías
15:    para cada (j, [clave, ·]) en T.entradas hacer insertar NODOCADENA(clave, j) al frente
       de su casilla
```

El borrado desenlaza el eslabón (recordando el anterior), marca la entrada como borrada y, si las entradas borradas superan a las vivas, compacta con un rehash al mismo m. Recorrer la tabla es recorrer `_entradas` salteando las borradas, por lo que **las claves se enumeran siempre en orden de inserción**. Esto hace que la construcción del grafo, Kosaraju y la simulación sean deterministas: dado el mismo escenario, producen exactamente el mismo resultado.

3.4 Parámetros

- **Factor de carga máximo** $\alpha = n/m \leq 0{,}75$**:** mantiene las cadenas cortas.
- **Crecimiento al primo siguiente a** $2m$**:** duplicar garantiza costo amortizado constante (Teorema 3); que m sea primo evita que las claves con un patrón regular se concentren en pocas casillas al reducir módulo m. La sucesión de tamaños es $17 \to 37 \to 79 \to 163 \to 331 \to 673 \to 1361 \to 2729 \to \cdots$
- **Semilla fija:** el sorteo de a, b se hace con un generador pseudoaleatorio con semilla constante (318), para que el programa sea reproducible. Se eligió evaluando las semillas 1 a 399 sobre las coordenadas de todos los escenarios y tomando una cuya cadena máxima fuera 2 en todos ellos (Tabla 1).

4 Algoritmo de Construcción

El procedimiento completo está en `GraphBuilder.construir_warehouse_desde_dict`. Previamente, la aplicación valida el JSON con `validar_escenario`: tipos y rangos, identificadores únicos, rectángulos dentro del mapa, obstáculos sin superponerse (los pasillos sí pueden cruzarse), racks con un lado de 2, puertas en el borde, zonas adyacentes a su puerta, productos dentro de su rack sin repetir $\langle x, y, z \rangle$, pedidos y pickers coherentes, y al menos un despacho con puerta de salida. Un JSON inválido produce un **error** y no se construye nada. En cambio, las anomalías que detecta la construcción producen **advertencias**: el almacén se construye igual y la interfaz las informa.

Algoritmo 2 CONSTRUIR-ALMACÉN(J)

Entrada: escenario J ya validado
Salida: `Warehouse` con $G = (V, E)$, accesos y lista de advertencias

$wh \leftarrow$ almacén vacío de $W \times H$; $ocupado \leftarrow \emptyset$; $decl \leftarrow \emptyset$; $adv \leftarrow [\,]$
para cada pasillo P de J, en orden **hacer** ▷ Paso 1: sentidos declarados
  para cada celda c del rectángulo de P **hacer**
    OCUPAR(c, $P.id$)
    si $decl[c] \neq$ AMBOS **entonces** $decl[c] \leftarrow (P.sentido,\ P.id)$ ▷ en un cruce, AMBOS prevalece
para cada puerta, luego zona, luego rack X de J **hacer** ▷ Pasos 2–4: obstáculos
  para cada celda c de X **hacer** OCUPAR($c, X.id$); $wh.obstaculos[c] \leftarrow$ celda del tipo de X
  registrar el objeto X en su diccionario; si X es rack sin lado 2, advertencia
para cada producto p de J **hacer** ▷ Paso 5: catálogo y altura ($z = 1$ por omisión)
  $wh.catalogo[p.id] \leftarrow p$; **si** $wh.obstaculos[(p.x, p.y)]$ es `CeldaRack` **entonces** apilar p en el nivel $p.z$
CONSTRUIR-DÍGRAFO(wh, $decl$) ▷ Pasos 6–8
$adv \leftarrow adv \parallel$ VALIDAR(wh) ▷ Paso 9: Kosaraju + controles
cargar configuración, pedidos y pickers; reubicar cada picker en un nodo ▷ Paso 10
devolver wh

procedimiento OCUPAR(c, id)
  si $c \notin \Omega$ **entonces error**: fuera del almacén
  si $ocupado[c]$ existe y es distinto de id **entonces** advertencia "id pisa a $ocupado[c]$"
  $ocupado[c] \leftarrow id$

Algoritmo 3 CONSTRUIR-DÍGRAFO(wh, $decl$)

para $x \leftarrow 0$ **hasta** $W - 1$ **hacer**
  para $y \leftarrow 0$ **hasta** $H - 1$ **hacer** ▷ Paso 6: nodos
    si $(x, y) \notin wh.obstaculos$ **entonces**
      $(s, id) \leftarrow decl[(x, y)]$ si existe, si no (AMBOS, NIL)
      $nodos[(x, y)] \leftarrow$ NODOGRAFO(x, y, s, id); $ady[(x, y)] \leftarrow [\,]$
para cada $u \in nodos$, en orden de inserción **hacer**
  para cada $\delta \in \Delta$ **hacer** ▷ Paso 7: accesos
    $o \leftarrow wh.obstaculos[u + \delta]$
    si o es celda del rack R **y** $-\delta \in \mathcal{D}_R(u + \delta)$ **entonces** registrar u como acceso de $u + \delta$
    si no si o es celda de la zona Z **entonces** registrar u como acceso de Z
  para cada $\delta \in D(\sigma(u))$ **hacer** ▷ Paso 8: aristas
    $v \leftarrow u + \delta$
    si $v \in nodos$ **y** $\neg\,\mathrm{CM}(\delta, \sigma(v))$ **y** $v \notin ady[u]$ **entonces** agregar v a $ady[u]$

Reubicación de pickers (Paso 10). Si la posición inicial ya es un nodo, se conserva. Si es una celda de rack, se usa su acceso principal. Si es una celda de zona, se usa el acceso de la zona más cercano en distancia Manhattan. En cualquier otro caso se conserva la posición informada (la validación ya emitió la advertencia).

4.1 Ejemplo Desarrollado

Se ejecutó el algoritmo sobre un almacén de 6×5 con:

- pasillo `PAS_IZQ` ($x = 1$, sentido N), pasillo `PAS_DER` ($x = 4$, sentido S) y dos pasillos horizontales AMBOS en $y = 0$ y $y = 4$;
- un rack de 2×3 en $(2, 1)$, una zona de 1×2 en $(5, 1)$ y su puerta en $(5, 3)$;
- dos productos, en $(2, 2, 1)$ y $(3, 1, 2)$.

Resultan 9 obstáculos y $|V| = 30 - 9 = 21$ nodos. Las celdas $(0, 1)$, $(0, 2)$ y $(0, 3)$ no están declaradas y quedan con sentido AMBOS. En los cuatro cruces, el pasillo horizontal AMBOS se impone y se registra una advertencia "pisa" por cada uno. Se generan $|E| = 42$ aristas. Por ejemplo:

- $(1, 1)$ es N: $D(\mathsf{N})$ = arriba, izquierda, derecha. La derecha es el rack, así que $ady[(1, 1)] = [(1, 2), (0, 1)]$.
- $(4, 0)$ es AMBOS, pero subir a $(4, 1)$, que es S, es contramano. Por lo tanto $ady[(4, 0)] = [(5, 0), (3, 0)]$.
- El nodo $(2, 0)$ toca al rack por debajo, pero el rack es vertical, de modo que esa es su punta y $(2, 0)$ no es acceso.

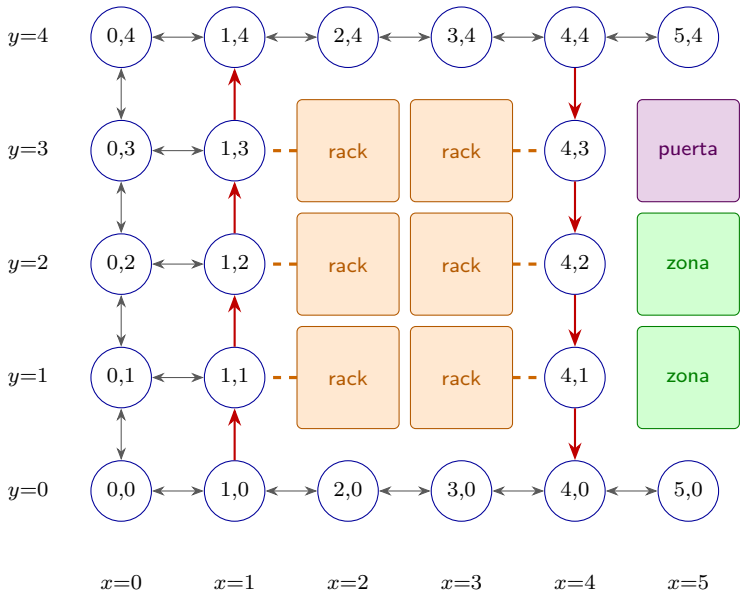


Figure 1: Dígrafo del ejemplo. Gris: pares con ida y vuelta. Rojo: aristas de un solo sentido (la columna $x = 1$ sube y la $x = 4$ baja). Naranja punteado: celda de rack → su acceso; la columna izquierda del rack se accede desde $x = 1$ y la derecha desde $x = 4$.

En la tabla de nodos, la inserción número 13, que corresponde a $(3, 0)$, supera $0{,}75 \cdot 17 = 12{,}75$ y dispara un rehash a $m = 37$. Al final, $\alpha = 21/37 = 0{,}57$ y hay exactamente dos colisiones: la casilla 9 contiene la cadena $(1, 2) \to (0, 1)$ y la casilla 17, la cadena $(5, 0) \to (0, 2)$. En cada cadena, el último insertado queda al frente.

5 Validación por Componentes Fuertemente Conexas

Con manos únicas pueden aparecer *trampas* (zonas a las que se entra y de las que no se sale) o *islas* (zonas a las que no se llega). Ambas situaciones se detectan con las componentes fuertemente conexas (CFC) de G.

Algoritmo 4 KOSARAJU-ITERATIVO(G)

```
visitado ← ∅;  fin ← [ ]
para cada s ∈ V en orden de inserción, s ∉ visitado hacer                ▷ 1er DFS sobre G
    visitado ← visitado ∪ {s};  pila ← [(s, 0)]
    mientras pila ≠ ∅ hacer
        (u, i) ← tope de pila
        si i < |ady[u]| entonces
            reemplazar el tope por (u, i + 1);  v ← ady[u][i]
            si v ∉ visitado entonces visitado ← visitado ∪ {v};  apilar (v, 0)
        si no desapilar;  agregar u al final de fin                      ▷ u terminó
G^T ← tabla hash con G^T[v] = [ u : (u, v) ∈ E ]                         ▷ grafo transpuesto
asignado ← ∅;  comps ← [ ]
para cada r en fin recorrido de atrás hacia adelante, r ∉ asignado hacer ▷ 2do DFS sobre G^T
    C ← [ ];  asignado ← asignado ∪ {r};  pila ← [r]
    mientras pila ≠ ∅ hacer
        u ← desapilar; agregar u a C
        para cada v ∈ G^T[u], v ∉ asignado hacer asignado ← asignado ∪ {v}; apilar v
    agregar C a comps
devolver comps ordenadas de mayor a menor tamaño
```

El primer DFS guarda en la pila el índice del próximo sucesor a explorar. Así simula la recursión sin usar la pila de llamadas de Python, cuyo límite (unas mil llamadas) se superaría en almacenes de 200×200 con caminos de decenas de miles de celdas.

Validar. La componente de mayor tamaño se toma como *red principal* C^*. Luego:

1. para cada celda de rack con algún acceso en C^*, ese acceso pasa a ser su acceso principal;
2. si hay más de una CFC, se informa cuántas hay, cuántas celdas quedan fuera de C^* y en qué rango de coordenadas;
3. se advierte por cada producto sin acceso o sin acceso en C^*, por cada zona en la misma situación, y por cada picker cuya posición inicial no es nodo o no está en C^*.

Ejemplo de trampa. Si en el ejemplo de la Sección 4.1 el pasillo izquierdo se declara `S` y ocupa las columnas 0 y 1, ninguna columna permite subir. Kosaraju devuelve 8 componentes: las dos filas horizontales (6 celdas cada una), los tres pares $\{(0, y), (1, y)\}$ y las tres celdas sueltas de la columna $x = 4$. La validación informa que 15 celdas quedan fuera de la red principal, que ambos productos y la zona no tienen acceso en ella y que el picker arranca fuera de ella. Durante el desarrollo, este control detectó una trampa real de 41 celdas en `escenario_grande.json`, que se corrigió.

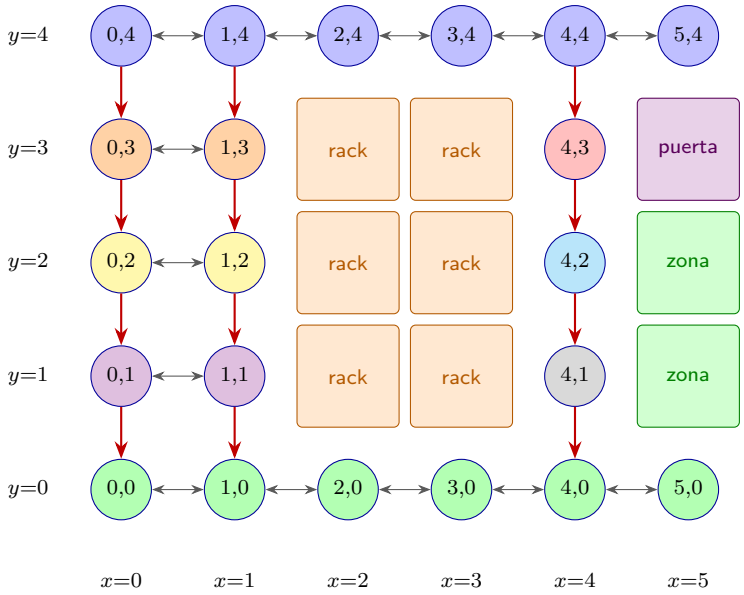


Figure 2: Escenario con trampa: cada color es una componente fuertemente conexa. Ninguna arista sube, por lo que desde abajo no se puede volver a la fila superior.

6 Taxonomía de Casos de Construcción

6.1 Caso 1: Cruce de Pasillos

Condición: una celda está cubierta por dos o más pasillos.
Resolución: los pasillos se procesan en el orden del JSON. Si la celda ya tiene AMBOS, lo conserva; si no, toma el sentido del pasillo actual. Así, si algún pasillo AMBOS la cubre, el cruce queda AMBOS y se puede doblar; si ninguno es AMBOS, queda el sentido del último declarado. Cada cruce genera además una advertencia informativa "pisa".

6.2 Caso 2: Piso Libre no Declarado

Condición: una celda no es obstáculo ni pertenece a ningún pasillo.
Resolución: es nodo con $\sigma = \text{AMBOS}$ y sin identificador de pasillo. Un escenario sin pasillos declarados es, por lo tanto, una grilla completamente bidireccional.

6.3 Caso 3: Obstáculo sobre un Pasillo o sobre Otro Obstáculo

Condición: un rack, zona o puerta ocupa una celda ya ocupada.
Resolución: se emite la advertencia "pisa". Como los nodos se crean consultando primero la tabla de obstáculos, la celda queda como obstáculo y no como nodo. (El validador previo ya rechaza obstáculos superpuestos entre sí.)

6.4 Caso 4: Forma del Rack

Condición: rack vertical ($a_R = 2$), horizontal ($h_R = 2$), cuadrado de 2×2 o sin ningún lado igual a 2.
Resolución: se aplica $\mathcal{D}_R$ de la Sección 2.2. El rack cuadrado se accede por cualquier lado. El rack sin lado 2 es rechazado por el validador; si llegara a la construcción, emite una advertencia y se accede por cualquier lado.

6.5 Caso 5: Trampas e Islas

Condición: G tiene más de una CFC.
Resolución: el almacén se construye igual, se advierte y se priorizan los accesos dentro de la red principal. Los módulos de ruteo trabajan sobre el mismo grafo, por lo que nunca se elige como destino un acceso al que no se podría llegar si existe otro que sí sirve.

6.6 Caso 6: Producto o Zona sin Acceso

Condición: todos los vecinos del costado largo de una celda de rack son obstáculos, o ninguno está en C^*; o una zona no tiene pasillos adyacentes.
Resolución: advertencia específica por producto y por zona. El armado de waves puede consultarlo en $O(1)$ con `accesos_rack_a_pasillo`.

6.7 Caso 7: Picker sobre un Obstáculo

Condición: la posición inicial no es un nodo.
Resolución: la validación lo advierte con la posición original y luego el picker se reubica en el acceso correspondiente (Paso 10).

6.8 Caso 8: Datos Fuera de Rango o Mal Formados

Condición: campos faltantes, tipos incorrectos, elementos fuera del mapa o sentido inexistente.
Resolución: error con mensaje claro (del validador o de OCUPAR); no se construye un almacén inconsistente.

7 Demostraciones y Análisis Asintótico

7.1 Demostración 1: Inyectividad de la Codificación de Coordenadas

Lema 1. *Si $0 \le y, y' < B$ y $0 \le x, x'$ con $xB + y < p$, entonces $k(x, y) = k(x', y') \Rightarrow (x, y) = (x', y')$. En particular, la codificación (1) es inyectiva sobre todas las coordenadas válidas ($x, y \le 199$).*

Proof. Si $xB + y = x'B + y'$, reduciendo módulo B se obtiene $y \equiv y' \pmod{B}$ y, como $0 \le y, y' < B$, resulta $y = y'$. Entonces $xB = x'B$ y $x = x'$. Para coordenadas válidas, $k \le 199 \cdot 1\,000\,003 + 199 < 2^{61} - 1 = p$, por lo que la reducción módulo p no altera el valor y la composición sigue siendo inyectiva. □

Gracias a este lema, dos celdas distintas nunca comparten clave entera. Las únicas colisiones posibles provienen de la reducción módulo m de la Etapa 2, y son las que resuelve el encadenamiento.

7.2 Demostración 2: Universalidad de la Familia $h_{a,b}$

Teorema 1 (Carter y Wegman; CLRS, Teorema 11.5)**.** *Sea p primo y $\mathcal{H} = \{h_{a,b} : a \in \mathbb{Z}_p^*,\ b \in \mathbb{Z}_p\}$. Para todo par de claves distintas $k \neq l$ en $\mathbb{Z}_p$,*

$$\Pr_{a,b}\big[h_{a,b}(k) = h_{a,b}(l)\big] \le \frac{1}{m}.$$

Proof. Sean $r = (ak + b) \bmod p$ y $s = (al + b) \bmod p$. Entonces $r - s \equiv a(k - l) \pmod{p}$. Como p es primo, $a \not\equiv 0$ y $k \not\equiv l$, el producto no es nulo módulo p, así que $r \neq s$: **antes de reducir módulo m no hay colisiones**.

Además, el sistema $r \equiv ak + b$, $s \equiv al + b$ tiene solución única $a = (r - s)(k - l)^{-1}$, $b = r - ak$ en $\mathbb{Z}_p$. Por lo tanto, la función $(a, b) \mapsto (r, s)$ es una biyección entre los $p(p-1)$ pares con $a \neq 0$ y los $p(p-1)$ pares (r, s) con $r \neq s$. Elegir (a, b) uniformemente equivale entonces a elegir (r, s) uniformemente entre los pares con $r \neq s$.

Hay colisión si y solo si $r \equiv s \pmod{m}$. Fijado r, la cantidad de valores $s \neq r$ en $\{0, \ldots, p-1\}$ con $s \equiv r \pmod{m}$ es a lo sumo $\lceil p/m \rceil - 1 \leq (p-1)/m$. Entonces la probabilidad de colisión es a lo sumo $\frac{(p-1)/m}{p-1} = \frac{1}{m}$. □

Por el Lema 1, las claves de dos celdas distintas son enteros distintos menores que p, de modo que el teorema se aplica directamente. La consecuencia práctica es que no existe un conjunto de coordenadas que sea "malo" de antemano para la tabla, cosa que sí ocurre con el método de la división $k \bmod m$ ante claves con patrones regulares.

Observación. La garantía es probabilística sobre la elección de (a, b). Al fijar la semilla, la tabla es determinista; la elección de la semilla 318 se verificó empíricamente sobre todos los escenarios (Tabla 1).

7.3 Demostración 3: Costo Esperado Constante con Encadenamiento

Teorema 2. *Con una función elegida de una familia universal y factor de carga $\alpha = n/m$, la longitud esperada de la cadena que recorre una búsqueda es a lo sumo $1 + \alpha$. Como la tabla mantiene $\alpha \leq 0{,}75$, buscar, insertar y borrar cuestan $O(1)$ esperado.*

Proof. Sea k la clave buscada y, para cada otra clave l almacenada, sea X_l la variable indicadora de $h(l) = h(k)$. La cadena de la casilla de k tiene longitud $L = [k \in T] + \sum_{l \neq k} X_l$. Por linealidad de la esperanza y el Teorema 1,

$$\mathbb{E}[L] \leq 1 + \sum_{l \neq k} \Pr[h(l) = h(k)] \leq 1 + \frac{n}{m} = 1 + \alpha.$$

Calcular h cuesta $O(1)$ para coordenadas, y la búsqueda recorre a lo sumo esa cadena. Insertar es una búsqueda más un enlace al frente, y borrar es una búsqueda más un desenlace. □

Teorema 3. *Una sucesión de n inserciones, con rehash al primo siguiente a $2m$ cada vez que $n > 0{,}75m$, cuesta $O(n)$ en total. Es decir, cada inserción cuesta $O(1)$ amortizado.*

Proof. Cada rehash al pasar de m_i a $m_{i+1} \geq 2m_i$ reinserta $n_i \leq 0{,}75\, m_i + 1$ claves y crea m_{i+1} casillas: cuesta $O(m_{i+1})$. Como los tamaños al menos se duplican, $\sum_i m_{i+1} \leq 2\, m_{\text{final}}$. Además $m_{\text{final}} = O(n)$: por el postulado de Bertrand el primo siguiente a $2m$ es menor que $4m$, y el último rehash ocurrió con $m < \frac{4}{3}n$. El costo total de los rehash es entonces $O(n)$, y sumado al $O(1)$ esperado de cada inserción da $O(n)$. □

7.4 Demostración 4: Correctitud del Dígrafo Construido

Teorema 4. *Al terminar* CONSTRUIR-DÍGRAFO*, el conjunto de claves de* `nodos` *es exactamente V, y $v \in ady[u]$ si y solo si $(u, v) \in E$.*

Proof. Nodos. El primer doble ciclo recorre cada $c \in \Omega$ exactamente una vez y la inserta si y solo si $c \notin \mathcal{O}$, es decir, si $c \in V$.

Aristas, ($\Leftarrow$). Sea $(u, v) \in E$, con $v = u + \delta$. Al procesar u, el ciclo interno recorre todo $\delta \in D(\sigma(u))$; en ese momento $v \in nodos$ (todos los nodos se crearon antes) y $\neg\, \mathrm{CM}(\delta, \sigma(v))$, así que v se agrega si no estaba.

Aristas, ($\Rightarrow$). El único punto donde se agrega v a $ady[u]$ exige $\delta \in D(\sigma(u))$, $v \in nodos = V$ y $\neg\, \mathrm{CM}(\delta, \sigma(v))$, que son exactamente las condiciones de la definición de E.

Como se controla $v \notin ady[u]$ antes de agregar, no hay aristas repetidas, y por lo tanto $|ady[u]| \leq |D(\sigma(u))| \leq 4$. □

7.5 Demostración 5: Correctitud y Costo de Kosaraju

Teorema 5 (CLRS, Teorema 22.16)**.** *El Algoritmo 4 devuelve exactamente las componentes fuertemente conexas de G, en tiempo $\Theta(|V|+|E|) = \Theta(|V|)$ esperado, más $O(c \log c)$ para ordenar las c componentes por tamaño.*

Esquema. Correctitud. El DFS iterativo visita y finaliza los nodos en el mismo orden que el recursivo, porque el par (u, i) reproduce exactamente el estado de cada llamada. Por el lema de tiempos de finalización (CLRS, Lema 22.14), si $C \neq C'$ son CFC y existe una arista de C a C', entonces $f(C) > f(C')$, donde f es el máximo tiempo de finalización de la componente. Luego, el nodo de mayor finalización pertenece a una componente C_1 sin aristas entrantes desde otras componentes aún no asignadas. En G^T, las aristas de C_1 hacia afuera quedan invertidas, así que el DFS iniciado en ese nodo alcanza exactamente C_1 (las CFC de G y de G^T coinciden). Por inducción sobre las componentes ya asignadas, cada árbol del segundo DFS es una CFC.

Costo. Cada nodo se apila una vez en cada DFS y cada arista se examina una vez en cada DFS y una vez al construir G^T. Cada consulta a las tablas hash es $O(1)$ esperado (Teorema 2). El total es $\Theta(|V| + |E|)$ y, como $|E| \leq 4|V|$, $\Theta(|V|)$. El ordenamiento final agrega $O(c \log c)$, que es $O(1)$ en los escenarios válidos ($c = 1$). □

Lema 2 (Garantía para los módulos de ruteo)**.** *Si G tiene una sola CFC, para todo par de nodos u, v existe un camino dirigido de u a v y otro de v a u. En consecuencia, todo producto con acceso, toda zona con acceso y todo picker ubicado en un nodo son mutuamente alcanzables, y Dijkstra encuentra siempre una ruta finita.*

Proof. Es la definición de componente fuertemente conexa aplicada a $C^* = V$. Los accesos y las posiciones de los pickers son nodos de V. □

7.6 Demostración 6: Linealidad de la Construcción Completa

Teorema 6. *Sean $W \cdot H$ el área, A la suma de las áreas de los rectángulos declarados, P la cantidad de productos, n la de pedidos con L ítems como máximo y K la de pickers.* Construir-Almacén *cuesta $\Theta(W \cdot H + A + P + nL + K)$ en tiempo esperado. Como los obstáculos no se superponen y los pasillos solo se superponen en los cruces, $A = O(W \cdot H)$ y el costo es lineal en el área.*

Proof. Marcar pasillos y obstáculos visita A celdas en total. Cada celda visitada realiza $O(1)$ operaciones esperadas sobre tablas hash. Los productos cuestan $\Theta(P)$. El ciclo de nodos es $\Theta(W \cdot H)$. El de accesos y aristas es $O(|V| \cdot |\Delta|) = O(W \cdot H)$. Kosaraju es $\Theta(|V|)$. La validación recorre accesos ($O(W \cdot H)$), productos, zonas y pickers. Los pedidos cuestan $\Theta(nL)$ y los pickers $\Theta(K)$. El espacio está dominado por las tablas, de tamaño $\Theta(|V| + |E| + |\mathcal{O}|) = \Theta(W \cdot H)$, más los datos de negocio. □

8 Resultados Experimentales

Las mediciones se tomaron con el código del proyecto (mediana de 5 corridas; Python 3, sin librerías externas).

Escenario	$W \times H$	$\lvert V\rvert$	$\lvert E\rvert$	CFC	m	α	cad. máx.	construir
`escenario_basico`	18×16	230	692	1	331	0,69	2	6,2 ms
`gen_chico`	10×29	207	514	1	331	0,63	2	5,7 ms
`gen_mediano`	26×37	652	1678	1	1361	0,48	2	18,3 ms
`escenario_grande`	60×40	1898	6048	1	2729	0,70	2	56,0 ms
`gen_grande`	42×85	2396	6226	1	5471	0,44	2	86,2 ms

Table 1: Escenarios del proyecto: tamaño del dígrafo, estado de la tabla de nodos y tiempo de construcción completa. Ninguna cadena supera los 2 elementos y todos los escenarios forman una única CFC.

Para medir la escalabilidad se generaron almacenes de $n \times n$ con racks verticales separados por pasillos de mano única alternada y dos franjas horizontales AMBOS. Se comparó además la tabla propia contra el diccionario nativo de Python (reemplazándolo solo para la medición), verificando que ambos producen el mismo grafo.

n	celdas	$\lvert V\rvert$	construir (propia)	μs/celda	Kosaraju (propia)	construir (`dict`)	factor
25	625	331	9,5 ms	15,2	2,1 ms	2,5 ms	$\times$3,9
50	2 500	1 028	30,8 ms	12,3	6,2 ms	7,6 ms	$\times$4,1
100	10 000	3 856	127,4 ms	12,7	21,3 ms	37,0 ms	$\times$3,4
150	22 500	8 192	400,7 ms	17,8	46,8 ms	104,5 ms	$\times$3,8
200	40 000	14 128	787,3 ms	19,7	88,0 ms	210,3 ms	$\times$3,7

Table 2: Escalabilidad. Al multiplicar el área por 64, el tiempo se multiplica por 83, y el costo por celda se mantiene en el mismo orden (12–20 μs), lo que es consistente con $\Theta(W \cdot H)$. Kosaraju crece igual que $\lvert V\rvert$.

La tabla propia es entre 3,4 y 4,1 veces más lenta que `dict`, y ese factor se mantiene **constante** con el tamaño. Esto es lo esperado: ambas son $O(1)$ esperado por operación, pero `dict` está implementado en C y la tabla propia en Python. El costo absoluto (menos de un segundo para el máximo permitido de 200×200) se paga una sola vez al cargar el escenario. Las consultas posteriores son $O(1)$ esperado. La implementación se verifica con 6 pruebas específicas dentro de la suite del proyecto (60 pruebas, todas exitosas): equivalencia con un diccionario en 4 000 operaciones aleatorias incluyendo el orden de iteración, fórmula y determinismo de $h_{a,b}$, inyectividad de las coordenadas, encadenamiento ante colisiones forzadas, invariante $\alpha \leq 0{,}75$ con m primo, y que el dígrafo construido coincide con el esperado.

9 Cuadro Resumen de Complejidades Computacionales

Notación: $W \cdot H$ área del almacén, $A = O(W \cdot H)$ suma de áreas declaradas, V nodos de pasillo, $E \leq 4V$ aristas, P productos, n pedidos con hasta L ítems, K pickers, n_T claves de una tabla hash.

Operación / Algoritmo	**Tiempo**	**Espacio**	**Justificación**
Codificar coordenada y $h_{a,b}$	$\Theta(1)$	$O(1)$	Aritmética entera; inyectiva (Lema 1).
Buscar / insertar / borrar en tabla	$O(1)$ esp., $O(n_T)$ peor	$\Theta(n_T + m)$	Hashing universal + encadenamiento, $\alpha \leq 0{,}75$ (Teorema 2).
Rehash	$\Theta(n_T + m)$; $O(1)$ amort.	$\Theta(m)$	Duplicación a tamaño primo (Teorema 3).
Recorrer la tabla	$\Theta(n_T)$	$O(1)$	Arreglo de entradas en orden de inserción.
Marcar pasillos y obstáculos	$\Theta(W \cdot H)$ esp.	$\Theta(W \cdot H)$	Una operación de tabla por celda de cada rectángulo.
Crear nodos	$\Theta(W \cdot H)$ esp.	$\Theta(V)$	Barrido completo de la grilla.
Accesos y aristas	$\Theta(V)$ esp.	$\Theta(V + E)$	Cuatro vecinos por nodo, $E \leq 4V$ (Teorema 4).
Kosaraju iterativo	$\Theta(V + E) = \Theta(V)$	$\Theta(V)$	Dos DFS con pila propia + transpuesto ($+O(c \log c)$ al ordenar c CFC).
Validación	$O(V + P + K)$	$O(V)$	Pertenencia a C^* en $O(1)$ con conjunto.
Construcción completa	$\Theta(W \cdot H + A + P + nL + K)$	ídem	Suma de las etapas; lineal en la entrada.
Sucesores de un nodo	$O(1)$ esp.	—	Una consulta a `lista_adyacencia`.
Acceso de un producto	$O(1)$ esp.	—	Catálogo + `acceso_rack_a_pasillo`.

Table 3: Resumen asintótico de la capa de modelado.

10 Conclusión

El almacén se modela como un dígrafo ralo de celdas de pasillo, construido en tiempo lineal en el área, cuya regla de aristas codifica con precisión la circulación de mano única y los giros en los cruces. Los accesos a racks y zonas se precalculan respetando que los racks se operan por su costado largo, de modo que el resto del sistema resuelve en tiempo constante dónde detener a un picker. Toda la topología reside en una tabla hash propia con hashing universal, encadenamiento y redimensionamiento a tamaño primo. Esto cumple con la exigencia de implementar las estructuras principales, con garantías demostradas de costo esperado constante, orden de iteración determinista y cadenas de longitud máxima 2 en todos los escenarios. Finalmente, Kosaraju certifica que el mapa forma una única componente fuertemente conexa, lo que garantiza que el armado de waves y el ruteo con Dijkstra siempre trabajan sobre ubicaciones mutuamente alcanzables, y detecta trampas e islas antes de simular.